- *Microsoft.Extensions.Hosting.WindowsServices*—Adds support for running the application as a Windows Service. To enable the integration, call `UseWindowsService()` on your `IHostBuilder` in Program.cs.

These packages each add a single extension method to `IHostBuilder` that enables the appropriate integration when running as a systemd daemon or as a Windows Service. The following listing shows how to enable Windows Service support.

Listing 34.9 Adding Windows Service support to a worker service

```
IHost host = Host.CreateDefaultBuilder(args)          ❶
    .ConfigureServices((hostContext, services) =>     ❶
    {                                                 ❶
        Services.AddHostedService<Worker>();          ❶
    })                                                ❶
    .UseWindowsService()                              ❷
    .Build();

host.Run();
```

❶ Configures your worker service as you would normally

❷ Adds support for running as a Windows Service.

During development, or if you run your application as a console app, `UseWindowsService()` does nothing; your application runs exactly the same as it would without the method call. However, your application can now be installed as a Windows Service, as your app now has the required integration hooks to work with the Windows Service system. The following basic steps show how to install a worker service app as a Windows Service: